OPENROCKET MIT

Flight-computer API and architecture

Namespace refactor, runtime contracts and framework integration

Edition 6.4 / 11 October 2026

The runtime namespace is now `edu.mit.rocket_team`. The functional subpackages remain intact. The main translated sketch is `edu.mit.rocket_team.FC.RTFC`.

This report describes the implementation, the OpenRocket engine connections, measurement and clock contracts, controller ownership, recovery, telemetry, the graphical designer, and surrounding simulation improvements. All diagrams, tables and equations are native LaTeX; the diagrams are drawn with TikZ in this self-contained source file. No external figures are required.

Refactor scope	150 pre-existing Java files: the full firmware translation and the files introduced by the MIT improvements.
Documentation	1,560 explicit methods and constructors documented, including the new six-method audit utility.
Verification	148 core tests and 65 Swing/UI tests passed; the root application JAR was rebuilt.
Architecture	Nine UML and sequence views; all 36 top-level firmware classes are indexed.

The report records software behavior and verified virtual-time contracts. It does not qualify physical flight hardware, processor execution time, inter-board communications or actuator dynamics.

Contents

1 Scope and namespace migration **2**

2 Connection to the OpenRocket engine **3**

3 Per-run runtime composition **4**

4 Timing contracts **5**

5 Measurements and coordinate conventions **7**

6 Firmware controllers and physical outputs **8**

7 Recovery commands and actual deployment **9**

8 Telemetry, logging and logical hardware **10**

9 Designer, persistence and custom programs **11**

10 Results and adjacent simulation improvements **12**

11 Complete firmware class index **13**

12 Build, regeneration and verification **15**

13 Source map and limits of the model **16**

List of Figures

1 Engine integration. The existing extension and listener connect the renamed runtime to the physical simulation. 3
2 Run-local state and the boundaries between acquisitions, task execution and latched outputs. 4
3 Virtual scheduling: a command completed at the PWM instant is available only after that instant's latch. . . 6
4 Instrument adapters and injected payloads. Hollow triangles identify the two shared base classes. 7
5 Firmware controller hierarchy and the physical bridge. The predictor owns its fit samples; recovery uses a separate event bridge. 8
6 A logical firing command and a physical deployment are distinct events. Engine eligibility checks remain authoritative. 9
7 Transmitted and received streams, plus the retained logical hardware adapters. 10
8 Portable FC definitions, simulation-specific assignments and the graphical editing/runtime connections. . . . 11
9 Result paths for FC diagnostics, trajectory animation, ensemble statistics and motor searches. 12

1 Scope and namespace migration

The former `edu.mit.rocket_team.zephyrus` root has become `edu.mit.rocket_team`. Subpackages such as `FC`, `instrument`, `control` and `telemetry` remain. The change updates source paths, package declarations, imports, module exports, tests and live documentation references.

```
// Previous: edu.mit.rocket_team.zephyrus.FC.RTFC
import edu.mit.rocket_team.FC.RTFC;
```

There is no compatibility package at the old namespace. External source that imports these firmware classes must update its imports, and compiled clients must be rebuilt. Custom boards using the supported API `info.openrocket.core.simulation.flightcomputer.JavaBoardProgram` do not require an import change.

Firmware model identifiers, `.fc` schema versions, telemetry byte layouts, existing log prefixes and the extension class `ZephyrusFlightComputer` retain their identities. Saved design and simulation-extension references can therefore continue to load. Historical benchmark logs retain their meaning as records of the builds that produced them.

1.1 Readability and documentation coverage

The source manifest covers the translated firmware and Java files introduced for the FC designer, controller blocks, physical actuator and recovery bridges, sensor models, telemetry replay, ensembles, motor batches, 3D plotting, update/memory helpers, tests and standalone verification tools.

Methods use consistent four-space formatting with blank-line separation. Every explicit method and constructor in that manifest has a Javadoc description, including private helpers and anonymous-class callbacks. Compiler-generated record accessors and implicit constructors are not explicit source declarations.

Selected pre-existing OpenRocket integration classes are included in the browsable reference for context. They are outside the newly documented-method coverage requirement; malformed comments and stale links encountered during Javadoc generation were repaired. The application runtime did not acquire a new documentation dependency.

1.2 Behavior preservation and reading the diagrams

The parser comparison found identical executable token streams in all 150 original scoped files after ignoring comments/formatting and normalizing the namespace. This comparison concerns this refactor; previously implemented controller and recovery features retain their own functional changes.

Class diagrams show selected operations rather than reproducing the entire API. A hollow triangle points to a superclass or interface; a filled diamond marks an owning runtime relationship; ordinary arrows denote use or data flow. Sequence diagrams describe virtual ordering, not desktop wall-clock latency. The full method reference is generated in `doc/improvements/javadoc/`.

2 Connection to the OpenRocket engine

`Simulation` selects an engine. Both `BasicEventSimulationEngine` and `ModifiedEventSimulationEngine` use the existing listener lifecycle and advance a physical stepper. `SimulationStatus` carries physical state, conditions, active components and the result branch. The FC extension freezes and validates its external design, checks custom Java approval and installs one `FlightControllerSimulatorListener`.

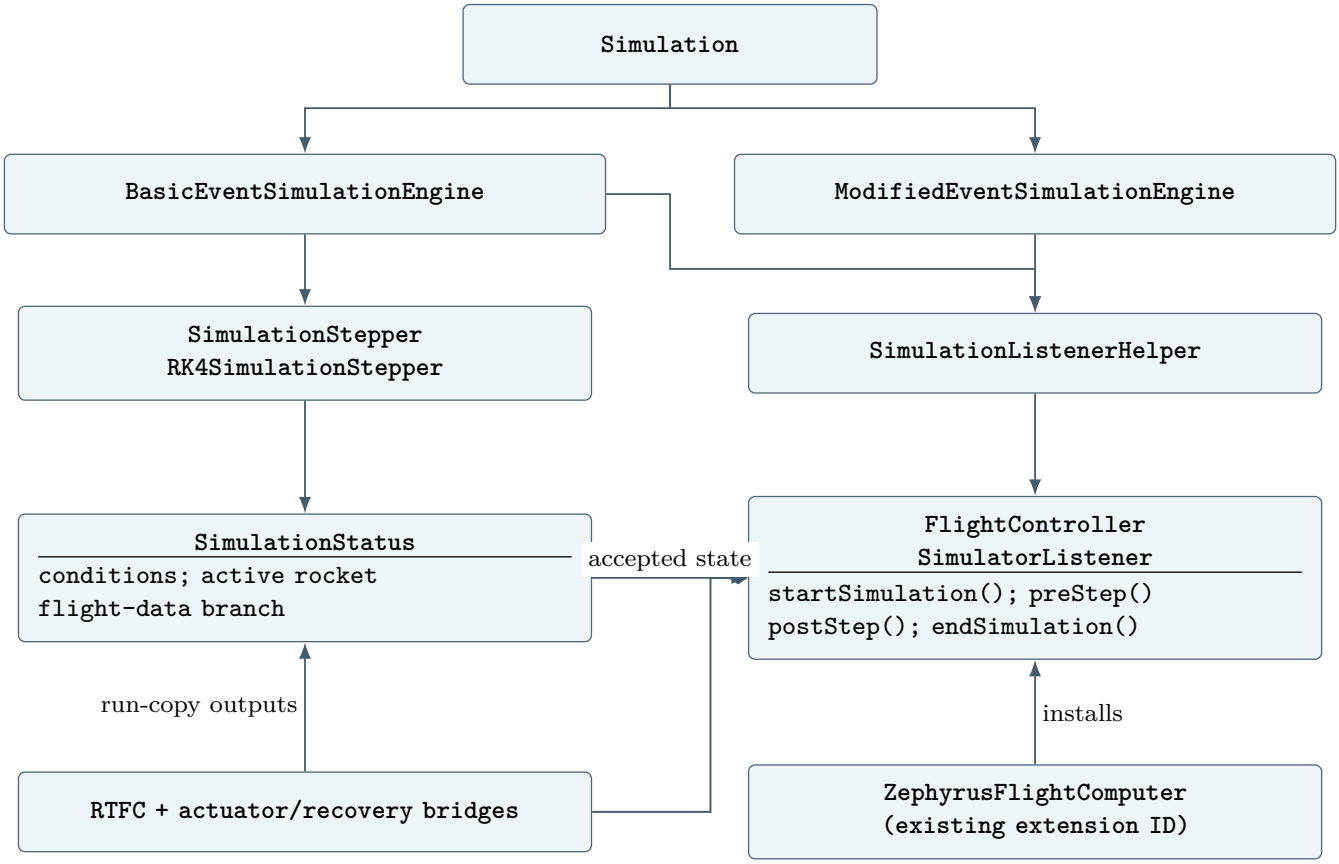


Figure 1: Engine integration. The existing extension and listener connect the renamed runtime to the physical simulation.

The listener creates a fresh `RTFC`, per-run context, telemetry outputs and physical bindings. Servo and cant changes apply to the simulation's rocket copy, preserving the user's design. `preStep` limits the next physical step to a virtual deadline. Coherent accepted acceleration/status samples feed acquisition; intermediate or future states are not replayed as earlier sensor measurements. `postStep` advances due work and synchronizes FC/physical result channels. Run shutdown closes outputs and pending recovery work. The separate `AirbrakesControllerListener` is not part of this FC path.

3 Per-run runtime composition

`RTFC` owns the translated instruments and firmware controllers. Design-defined state graphs, controller blocks and Java boards extend that runtime. `RTFC.Inputs` carries engineering-unit acquisitions and their timestamps. `RTFC.Output` is the PWM-latched snapshot, including legacy channels, named servo maps and per-controller outputs.

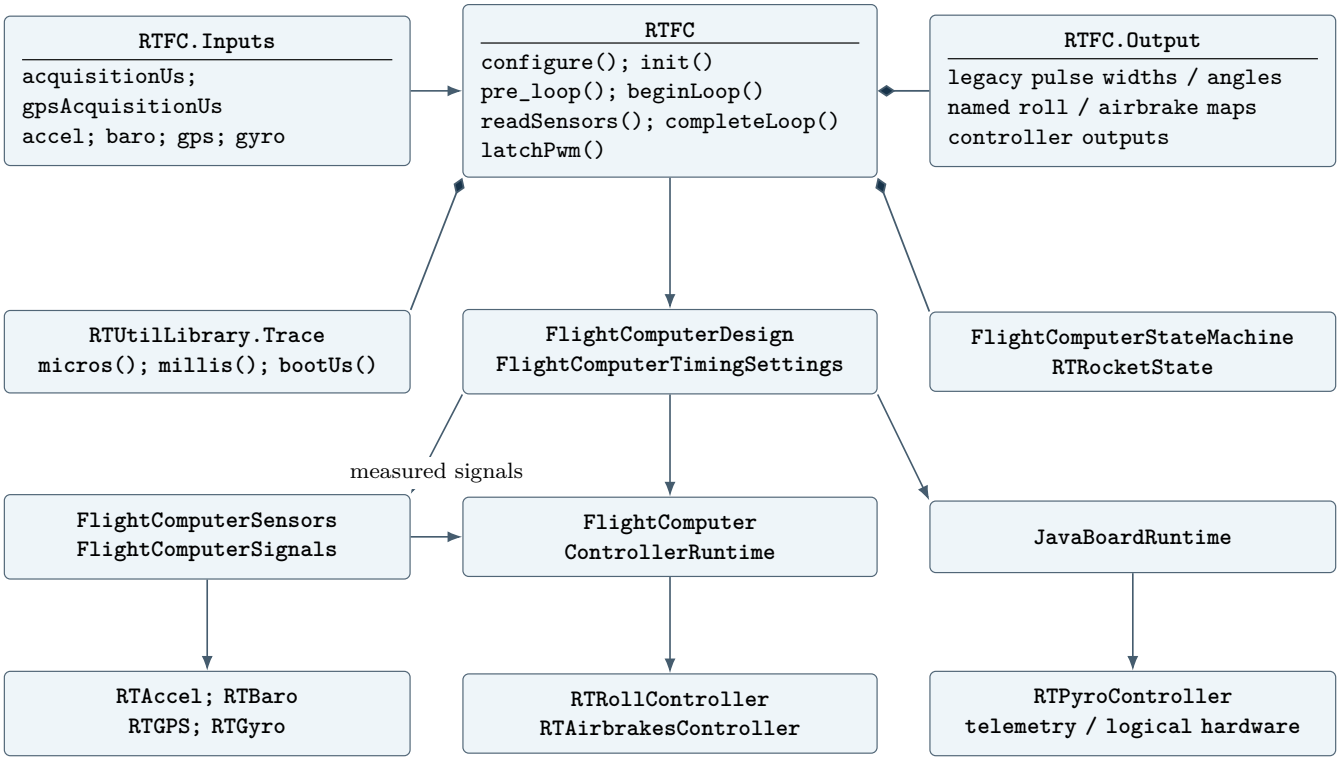


Figure 2: Run-local state and the boundaries between acquisitions, task execution and latched outputs.

The trace is a virtual boot clock and action sink shared by the run's devices. Sensor histories, task instances, controller state and channel caches belong to the run. A new simulation receives fresh instances. Physical comparison curves are recorded separately from the measurements used by controllers. Logical commands and PWM-latched physical commands are separate states.

4 Timing contracts

Boot timestamps are integer microseconds. Physical simulation time zero follows 1,000,000 μs of virtual warm-up. The preflight command is queued at 500,000 μs during warm-up. Firmware `micros()` and `millis()` retain unsigned 32-bit wrap; the trace additionally stores unwrapped boot time. For clocks in the same units, the firmware elapsed-time operation is

$$\Delta t_{32} = (t_{\text{now}} - t_{\text{then}}) \bmod 2^{32}. \tag{1}$$

Boundary	Implemented contract
FC loop	Nominal 10,000 μs period, with modeled sensor-read cost, extra work and seeded execution jitter. Loop phases cannot overlap.
Held acquisition	Input acquisition and availability times are distinct. A loop uses its held acquisition; GPS retains the original fix timestamp.
Independent tasks	Java boards and controller blocks freeze inputs at start, publish at completion and follow independent periods and execution costs.
PWM	Commands latch every 20,000 μs with configurable phase. A task completing exactly on an interrupt boundary misses that latch.
State epochs	State changes invalidate work begun in an earlier state; explicit controller-close actions invalidate pending block work.
Recovery	Accepted commands retain completion time in microseconds. Physical deployment deadlines do not wait for servo PWM or radio delivery.

Table 1: Virtual contracts enforced by the existing listener scheduler.

The firmware's nominal wait is millisecond-quantized. For loop start s_n and completion c_n, both in boot microseconds,

$$s_{n+1} = \max\Big(c_n,\ 1000\left(\left\lfloor \frac{s_n}{1000} \right\rfloor + 10\right)\Big). \tag{2}$$

A custom board or controller block instead uses its own configured period P:

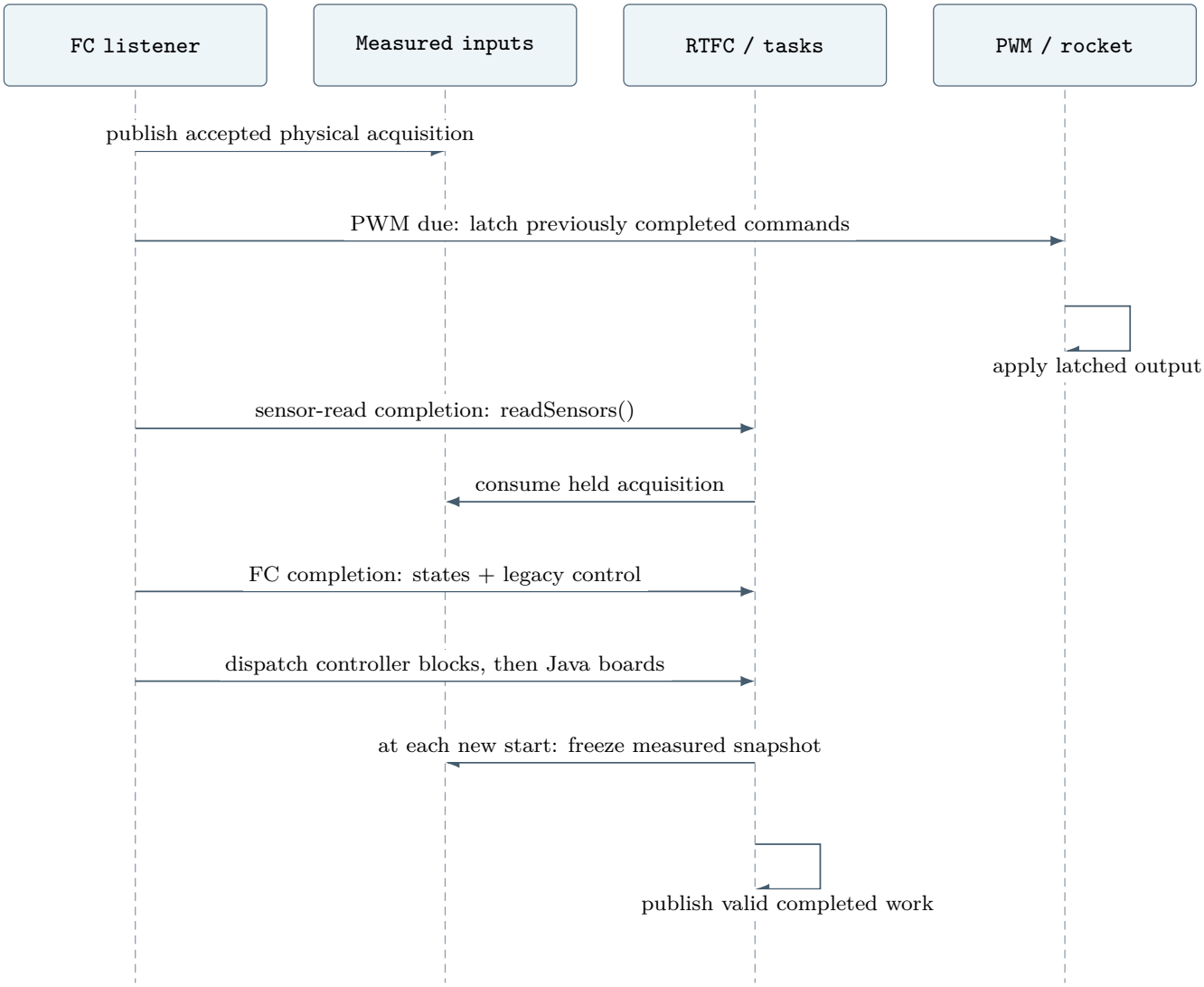
$$s_{n+1} = \max(s_n + P,\ c_n). \tag{3}$$

An overrun delays the next invocation; it does not create overlapping calls or a catch-up burst. A faster task does not create fresher measurements.

These are virtual-time contracts. The desktop's elapsed execution time is not the modeled processor time. Execution costs remain supplied approximations; board grouping does not simulate shared CPU arbitration or communications contention between physical boards.

4.1 Dispatch ordering and the PWM completion tie

At a coincident boundary, PWM latching precedes FC completion, then controller and Java-board work. Figure 3 shows the relevant ordering; GPS availability and other due services remain part of the same dispatcher.



A command published in the lower part of this sequence cannot alter the earlier PWM latch. It becomes physical at a later interrupt. No wall-clock sleep defines these deadlines.

Figure 3: Virtual scheduling: a command completed at the PWM instant is available only after that instant's latch.

Task completion must also belong to the current state epoch. Work invalidated by a state change cannot publish stale control output. The engine's next physical step is capped at the next pending virtual deadline so the scheduler does not recreate missed loop ticks using future physical samples.

5 Measurements and coordinate conventions

`FlightComputerSensors` applies configured mounting rotation, bias, acquisition cadence and seeded Gaussian noise to accepted simulator inputs. `FlightComputerSignals` publishes stable source IDs, channel descriptors, validity and acquisition/availability times. Primary sensors retain firmware filtering and integration; additional sensors keep independent histories.

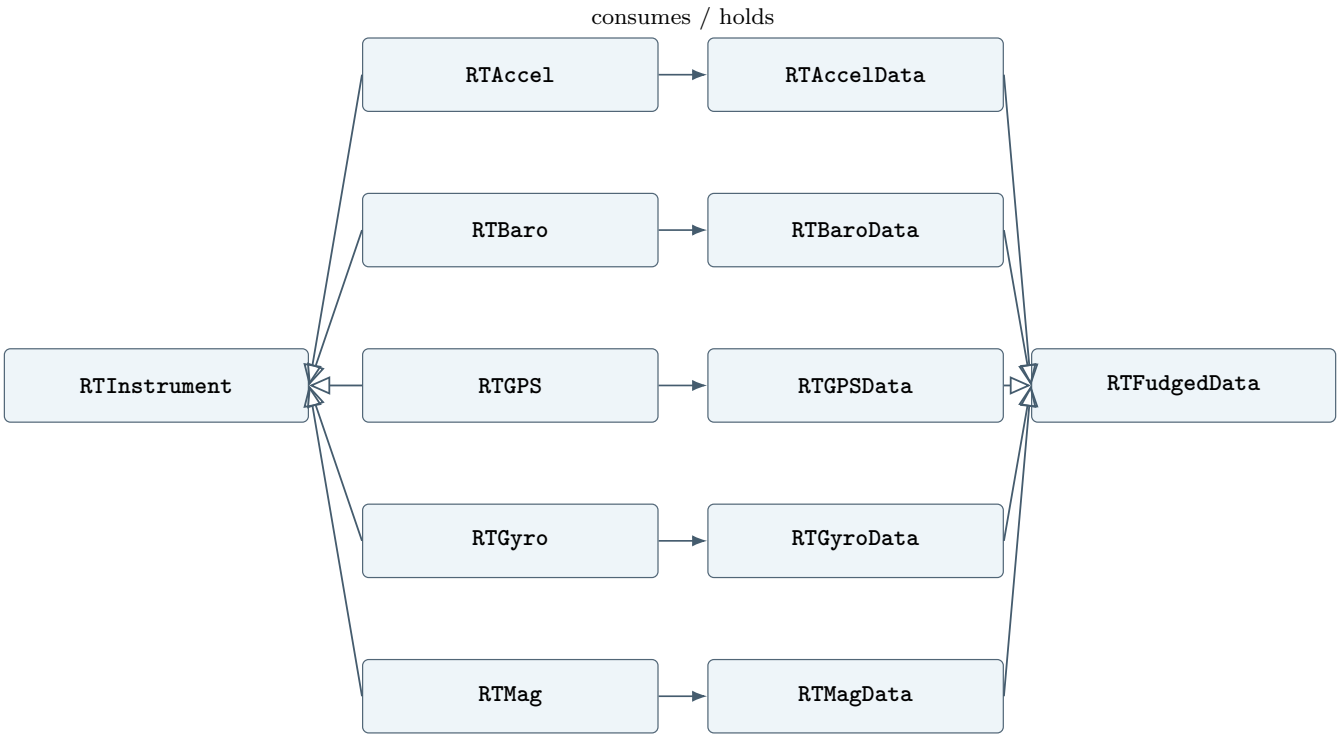


Figure 4: Instrument adapters and injected payloads. Hollow triangles identify the two shared base classes.

Quantity	Interface convention
Acceleration	Specific force in m/s^2; FC sensor X is OpenRocket body Z. The legacy velocity estimate integrates sensor X minus gravity, rather than ground-speed magnitude or airspeed.
Angular rate	Sensor rates in degrees/s. Roll integrates negative sensor X; pitch/yaw use the corresponding held rates.
Barometer	Pressure in hPa, temperature in Celsius and height in metres. Pressure altitude passes through the moving average and zero reference.
GPS	Input coordinates are decimal degrees and altitude is MSL metres; quantization is 10^{-7} degrees and millimetres. The adapter's `getHeight()` and `getAltitude()` return zero-relative height.
Raw counts	Pressure/temperature ADC counts may be NaN when unavailable. Engineering observations do not imply a modeled electrical sensor bus.

Table 2: Units at the measured-input boundary.

Controllers consume measurements; simulator truth is a separate diagnostic series. Re-reading a held signal does not draw fresh noise. `RTMag` is a legacy adapter outside the active FC measurement loop.

6 Firmware controllers and physical outputs

The translated roll controller retains its aerodynamic gain schedule and elapsed-time PID behavior. The airbrake controller retains the FC-included coast predictor and state sequence, including integer flight seconds and the source integral convention. Generic controller-block PID does not replace these legacy laws.

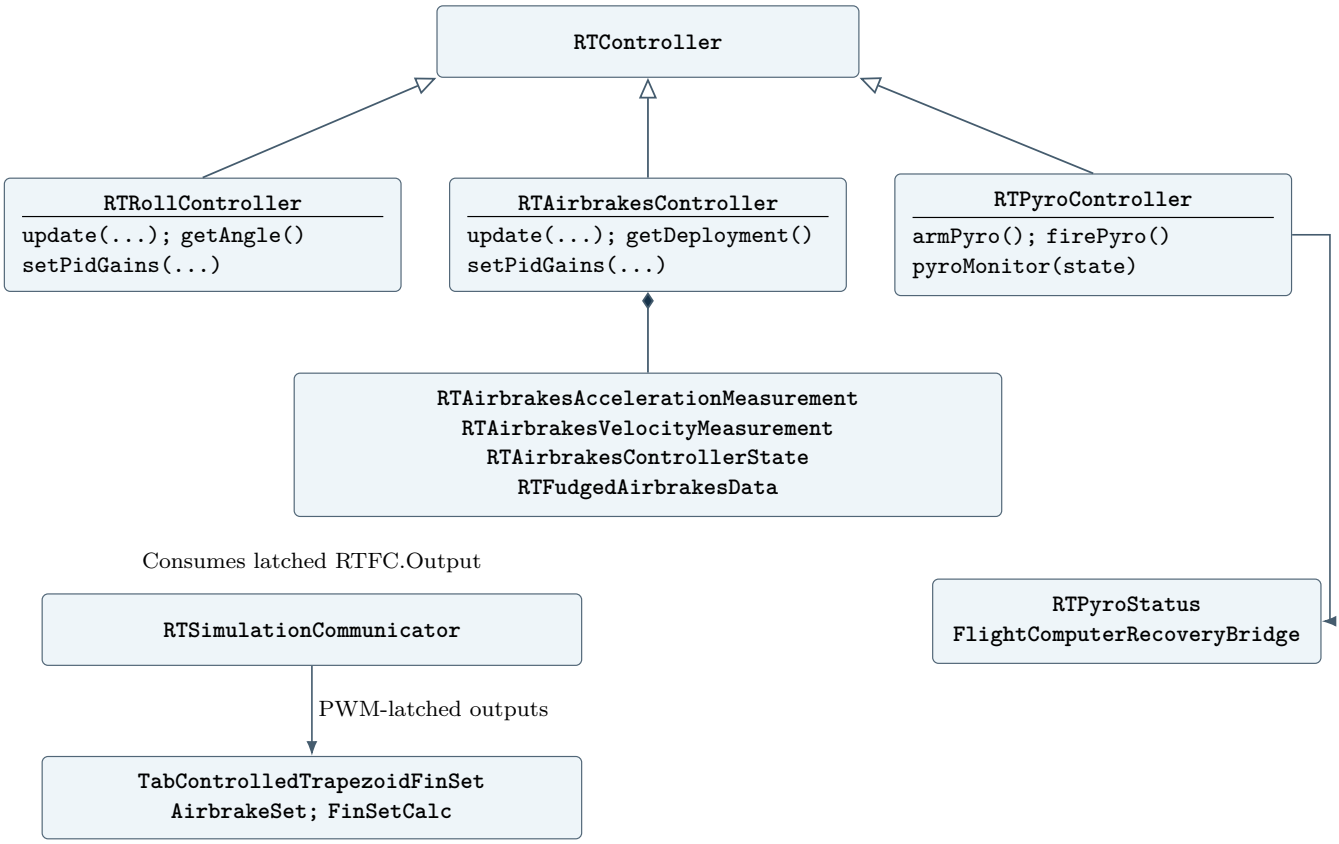


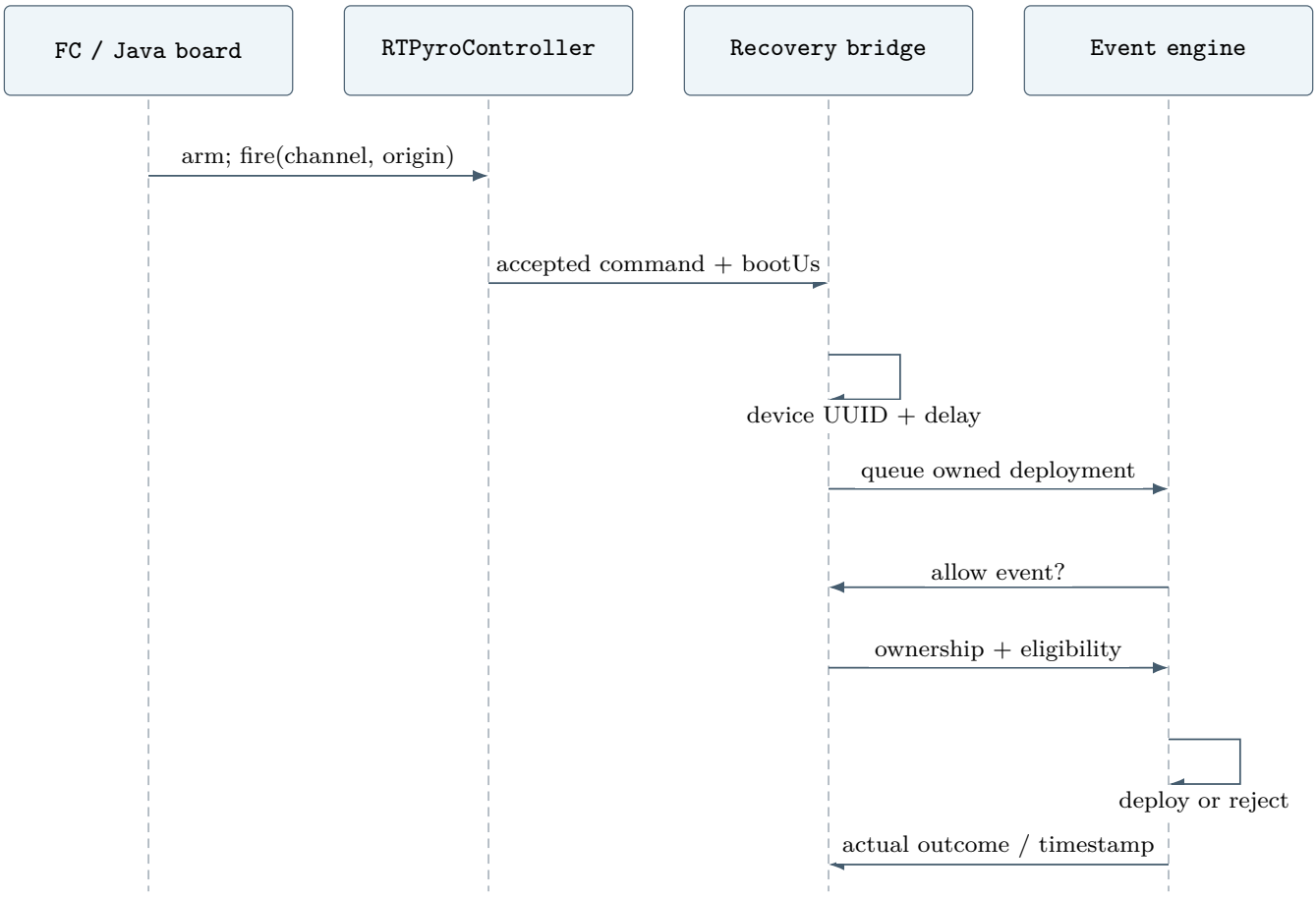
Figure 5: Firmware controller hierarchy and the physical bridge. The predictor owns its fit samples; recovery uses a separate event bridge.

`RTSimulationCommunicator` applies latched airbrake fractions and roll-tab angles to active components in the per-run rocket. FC angles are degrees; fin geometry uses radians. Named Java writes require exact, unique rocket component names. Missing FC hardware discards the associated command; present hardware without a suitable physical binding is diagnosed according to the binding rules.

Controller blocks use stable component UUIDs for fin tabs, fin cant and airbrakes. Enabled mapped blocks reserve their outputs even outside their active states, applying the selected hold/neutral policy. Java broadcasts skip reserved targets; conflicting explicit named writes are rejected. State-disjoint blocks may share targets under validated ownership rules. Actuator movement is ideal at a PWM latch: finite slew, inertia and backlash are not modeled.

7 Recovery commands and actual deployment

`RTPyroController` provides six logical channels, arming, firmware status and pulse timing. An accepted fire command records a sequence, origin and exact boot time. `FlightComputerRecoveryBridge` resolves channel-to-device UUID assignments and delay, then queues an FC-owned engine event.



Actual deployment is recorded separately from the pulse. FC recovery deadlines are independent of servo PWM and radio delivery. Listener vetoes and engine abort conditions can prevent physical deployment.

Figure 6: A logical firing command and a physical deployment are distinct events. Engine eligibility checks remain authoritative.

FC-enabled simulation overrides normal OpenRocket deployment rules. Those saved rules resume when FC simulation is disabled. Physical deployment is still subject to engine checks, listener vetoes and abort conditions, and is recorded separately for logs, plots and canopy animation.

The firmware pulse expires using a strict elapsed-millisecond threshold of more than 250 ms. Physical deployment scheduling retains the accepted command's microsecond timestamp. Continuity affects firmware status reporting; the `firePyro` acceptance gate itself checks arming rather than requiring simulated continuity.

8 Telemetry, logging and logical hardware

`RTTelemetryEngine` preserves transmitted packet bytes and decodes them to ground-station-compatible CSV. Received streams add deterministic packet loss and delivery delay. `OR.log` contains action records and structured physical/FC plot samples; metadata records paths, settings and completion.

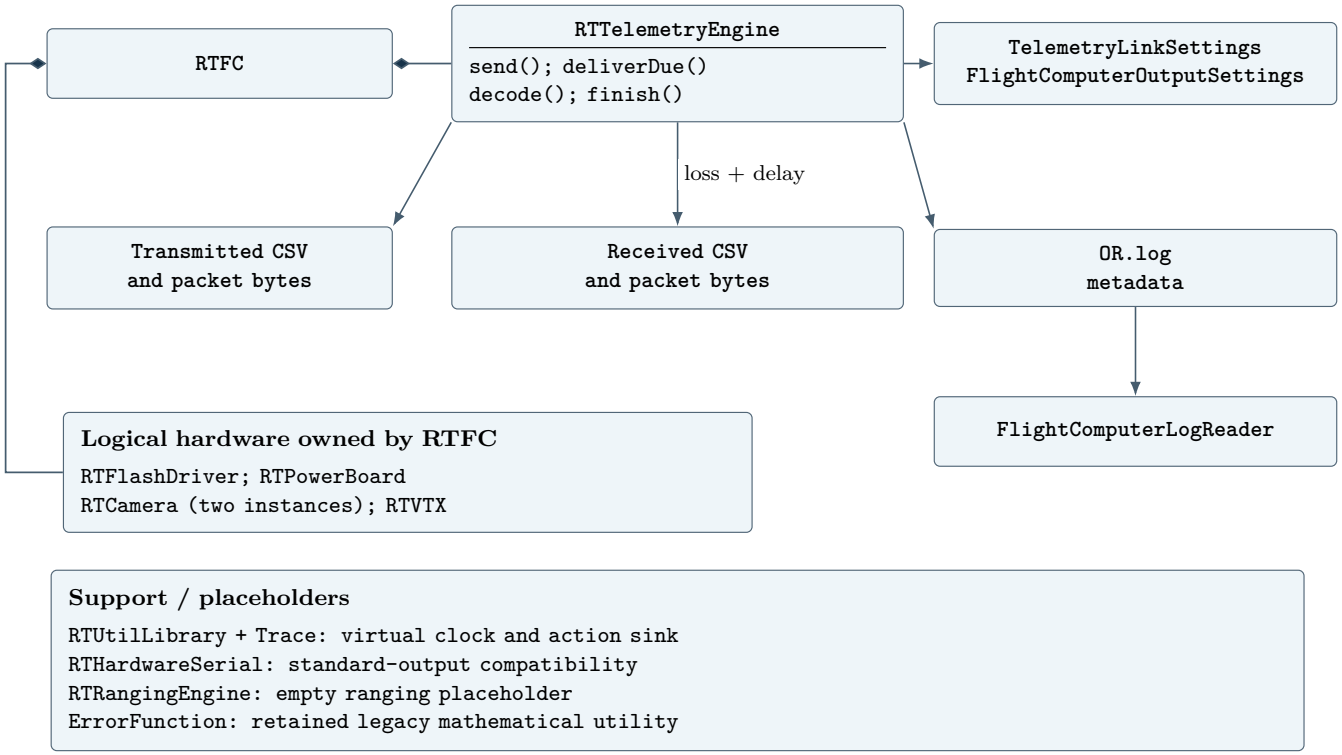


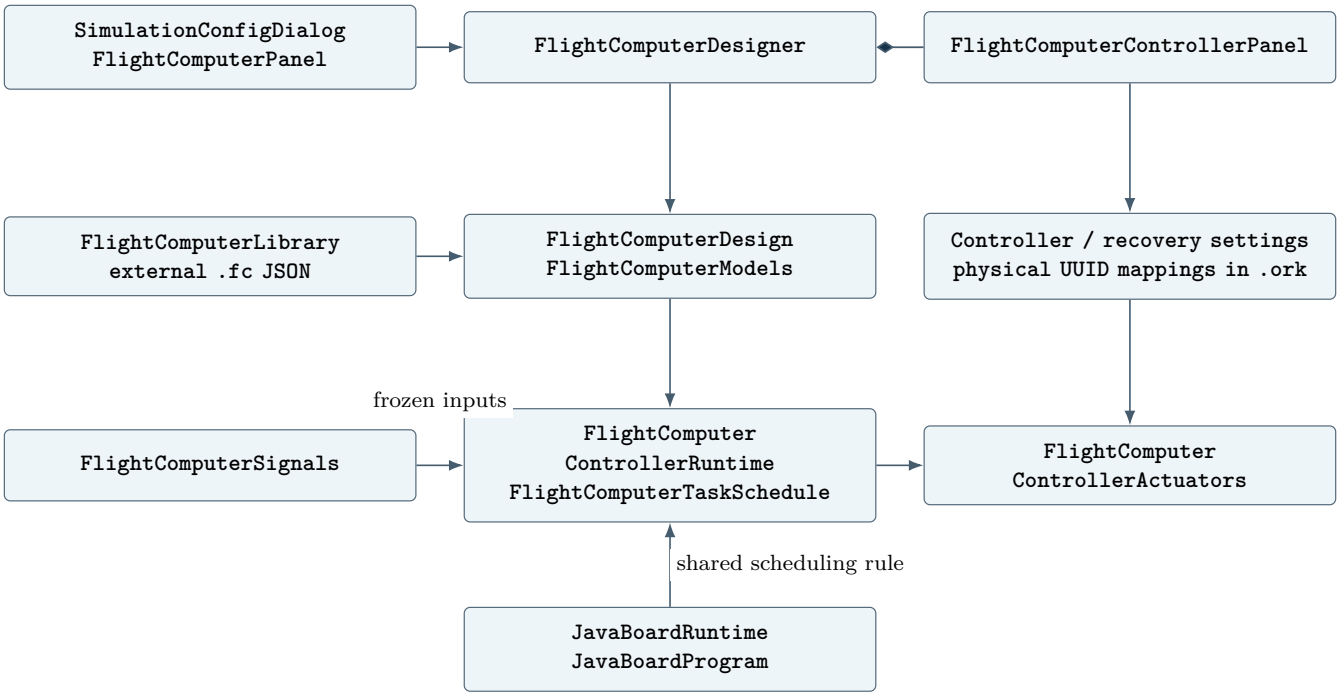
Figure 7: Transmitted and received streams, plus the retained logical hardware adapters.

Existing output files are protected against overwrite. Batch and ensemble members receive distinct output paths. Telemetry is streamed, while result branches retain plotted samples; short-period traces can still produce large files. Downlink effects change ground observability, not the closed-loop physical control calculation.

`FlightComputerLogReader` reconstructs available channels and actual recovery events without executing custom Java from an imported file. Camera/video, flash and power adapters expose logical operations or nominal values. They do not model RF propagation, electronics, flash wear or battery dynamics. The serial wrapper opens no physical serial port.

9 Designer, persistence and custom programs

`FlightComputerDesign` is the immutable JSON design and validation boundary. `FlightComputerModels` defines hardware capabilities and defaults. The standard library imports external `.fc` files, resolves references and records behavior fingerprints, excluding presentation-only names/layout from those fingerprints.



Portable definitions live in .fc. The .ork stores the external reference, result provenance and rocket-specific physical assignments. Custom Java requires approval of its exact source hash.

Figure 8: Portable FC definitions, simulation-specific assignments and the graphical editing/runtime connections.

The simulation's Flight computer tab opens the designer. Hardware, behavior, Java and controller editors apply snapshot changes with undo/redo. Controller editing combines algorithm settings, measured-source/quantity selection, baselines, tuning, timing, states and eligible physical targets.

Custom source derives from `JavaBoardProgram`; each run creates fresh instances. Compilation does not initialize the program class. Execution requires local approval of the exact source hash and occurs inside the OpenRocket process, without a sandbox. Independently scheduled boards run in tandem for algorithm experiments, but do not provide an authoritative model of inter-board communications.

10 Results and adjacent simulation improvements

`FlightComputerData` and `FlightComputerControllerPlots` record measured/physical comparisons, servo outputs, controller diagnostics and state. Digital commands/status use held-value plotting. Structured log metadata preserves channel identities and units during import.

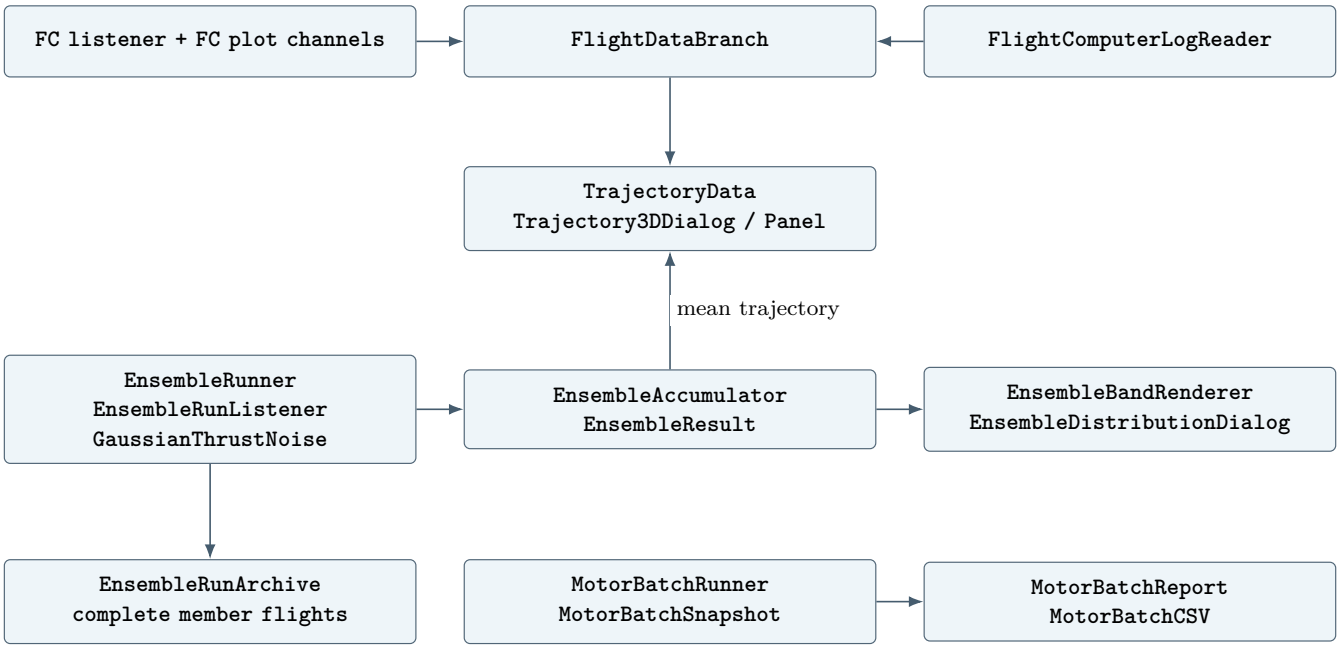


Figure 9: Result paths for FC diagnostics, trajectory animation, ensemble statistics and motor searches.

`TrajectoryData` reconstructs position, recorded quaternion orientation, motor burns and recovery events. The cartoon attitude follows that orientation, not velocity direction. Actual deployment controls the canopy, burn intervals control exhaust, and FC state labels follow the playhead.

Ensemble engine variation is additive:

$$T_{\text{member}}(t) = \max\{0,\, T(t) + e(t)\}, \qquad e(t) \text{ is the configured Gaussian perturbation during burn.} \tag{4}$$

Configured atmospheric variation is applied separately. Members are aligned in physical time to form the mean and sample standard deviation. 2D plots show mean $\pm 1\sigma$; 3D uses the mean trajectory. Scalar distributions use each member's extrema/metrics, and the archive retains complete flights.

Motor-batch workers use isolated baseline snapshots, record candidate/delay outcomes and export CSV. Update and memory helpers are documented maintenance utilities, rather than FC runtime dependencies.

11 Complete firmware class index

All entries below are beneath `edu.mit.rocket_team`. The generated Javadoc gives the complete method-level API, including private members. The index includes legacy and placeholder classes so their presence is not mistaken for active physical hardware simulation.

Class	Subpackage	Responsibility
`RTCamera`	`AV`	Logical camera setup; two instances are owned by the FC.
`RTVTX`	`AV`	Logical video-transmitter power requests.
`FlightComputerTimingSettings`	`FC`	Read/work/jitter costs, PWM phase and timing seed.
`RTFC`	`FC`	Translated sketch, loop phases, state actions and PWM snapshots.
`RTPyroController`	`control`	Six-channel arm/fire, pulse and status logic.
`RTRollController`	`control`	Firmware roll PID and aerodynamic gain schedule.
`RTAirbrakesAccelerationMeasurement`	`control.airbrakes`	Acceleration/time sample for the coast fit.
`RTAirbrakesController`	`control.airbrakes`	Included coast predictor and airbrake control sequence.
`RTAirbrakesControllerState`	`control.airbrakes`	Predictor preparation and control phase enumeration.
`RTAirbrakesVelocityMeasurement`	`control.airbrakes`	Velocity/time sample for the coast fit.
`RTAccel`	`instrument`	Fresh-sample consumption and longitudinal velocity integration.
`RTBaro`	`instrument`	Pressure altitude, moving average, zeroing and maximum tracking.
`RTGPS`	`instrument`	Held GPS fix, quantization, relative height and maximum tracking.
`RTGyro`	`instrument`	Angular-rate integration and firmware attitude conventions.
`RTMag`	`instrument`	Legacy magnetometer/roll injection adapter.
`RTFlashDriver`	`internal`	Logical logging allocation and packet append operations.
`RTPowerBoard`	`internal`	Nominal power values, converter enables and protection flags.
`FlightComputerOutputSettings`	`telemetry`	CSV/log paths and distinct multi-run filenames.
`RTRangingEngine`	`telemetry`	Reserved empty ranging placeholder.
`RTTelemetryEngine`	`telemetry`	Byte queues, loss/delay, packet decoding and streamed files.
`TelemetryLinkSettings`	`telemetry`	Reproducible downlink configuration.

Class	Subpackage	Responsibility (continued)
`ErrorFunction`	`util`	Legacy error-function and normal-distribution approximations.
`RTController`	`util`	Common setup, loop and injection interface for controllers.
`RTInstrument`	`util`	Instrument base sharing the run-local clock and trace.
`RTRocketState`	`util`	Firmware phase codes used by commands and telemetry.
`RTSimulationCommunicator`	`util`	Latched commands to per-run airbrakes and tab fins.
`RTUtilLibrary`	`util`	Unsigned clock arithmetic, nested Trace and orientation helpers.
`RTAccelData`	`util.data`	Mutable sensor-axis specific-force payload.
`RTBaroData`	`util.data`	Barometer engineering values and optional raw ADC fields.
`RTFudgedAirbrakesData`	`util.data`	Legacy predictor altitude/velocity/acceleration/apogee inputs.
`RTFudgedData`	`util.data`	Shared marker base for injected payloads.
`RTGPSData`	`util.data`	Coordinates, MSL altitude, fix and dilution-of-precision values.
`RTGyroData`	`util.data`	Mutable angular-rate payload.
`RTMagData`	`util.data`	Legacy magnetometer and supplied roll payload.
`RTPyroStatus`	`util.data`	Two-bit firmware continuity/firing status enumeration.
`RTHardwareSerial`	`util.hardware`	Standard-output compatibility for firmware serial calls.

12 Build, regeneration and verification

The code scope and context classes are defined by two manifests within the Gradle project:

```
config/mit-documentation-sources.txt
config/mit-javadoc-connectors.txt
```

From `clone/openrocket-release-24.12.RC.01`, with JDK 17:

```
./gradlew formatMitJava
./gradlew verifyMitJavadoc rocketTeamJavadoc
./gradlew shadowJar
```

Run formatting before compilation or documentation generation, not concurrently. The formatter is pinned to google-java-format 1.24.0 with four-space indentation and preservation of import order, unused imports and string literals. Documentation dependencies are developer-only.

`verifyMitJavadoc` uses the JDK parser to audit every explicit method and constructor in the improvement manifest, including private methods, test helpers and anonymous callbacks. It fails on a missing description. `rocketTeamJavadoc` includes private members and selected engine types, and checks comment syntax and links. Missing field descriptions in upstream classes are excluded from warnings; the separate method audit is not disabled.

12.1 Compiling this entirely LaTeX report

This file contains all prose, equations, tables and nine TikZ figures. There are no external image inputs, shell-escape requirements or PlantUML dependencies. From `doc/improvements`:

```
latexmk -pdf -interaction=nonstopmode \
  -halt-on-error 14-flight-computer-api-and-architecture.tex
```

A normal LaTeX distribution with TikZ, Latin Modern, hyperref, listings, booktabs, longtable and the packages listed in the preamble is sufficient. The existing Markdown guide and its `rocketTeamUml` SVG generation task remain separate companion documentation. This report's figures are edited in the TikZ source, not regenerated from those SVGs.

12.2 Recorded validation results

- **1,560/1,560 explicit methods and constructors documented** across 150 original improvement files and the new parser/audit utility.
- **150/150 original scoped files** retain their executable Java tokens after comment/spacing removal and namespace normalization.
- **148 core tests and 65 Swing/UI tests passed**, with no failures, errors or skipped tests. Coverage includes firmware replay, microsecond scheduling, PWM ties, recovery, independent actuators, controller blocks, ensembles, motor batches, plots, editor lifetime and update routing.
- **Root shadow JAR rebuilt successfully**. The startup class and `edu/mit/rocket_team/FC/RTFC.class` are present; no entries remain beneath the old namespace.
- **Javadoc generated successfully** with documentation/link checks. All 36 top-level firmware classes are represented by this report's figures and complete class index.

Application artifact: `build/libs/OpenRocket-MIT-v6.4.jar`, with its checksum and installed-app compatibility alias. Restart an already-running application to load the renamed classes.

13 Source map and limits of the model

The main integration entry points, relative to the Gradle project, are:

```
core/src/main/java/edu/mit/rocket_team/
core/src/main/java/info/openrocket/core/simulation/
  listeners/FlightControllerSimulatorListener.java
  extension/impl/ZephyrusFlightComputer.java
  flightcomputer/
swing/src/main/java/info/openrocket/swing/gui/
  flightcomputer/
  simulation/FlightComputerPanel.java
```

The connected framework types include `Simulation`, the two event engines, `SimulationStatus`, `SimulationConditions`, `SimulationOptions`, the stepper/listener interfaces and helpers, `FlightDataBranch`, `FlightEvent`, `FinSetCalc`, controlled fins, airbrakes and `RecoveryDevice`.

Area	What the implementation establishes
Virtual scheduling	Deterministic ordering, acquisition age, completion delay and PWM/recovery deadlines for the configured model.
Processor and buses	User-supplied execution costs; no authoritative shared-CPU or inter-board communications model.
Sensors	Engineering-unit acquisition, noise/bias, rotation and firmware filtering; not complete sensor electronics.
Servos and aerodynamics	Commands reach physical fin/airbrake models; no modeled servo dynamics or general qualification of high-deflection aerodynamics.
Recovery	Accepted logical commands connect to actual engine deployment events, subject to eligibility and abort rules.
Radio and power	Packet loss/delay and logical/nominal hardware behavior; not RF, battery or electrical dynamics.

Table 4: Implementation contracts and their practical limits.

13.1 Companion documents

Within `doc/improvements`, the recovery implementation is described in `11-flight-computer-recovery-implementation.md`; controller behavior, ownership, tuning and verification are described in `13-flight-computer-controller-implementation.md`. The browsable method reference begins at `javadoc/index.html`.

Repository: github.com/MDNich/ActiveControl_MIT_RktTeam. The report describes the 6.4 implementation verified on 11 October 2026. Diagrams are curated architecture views and must be updated when source relationships or scheduling order change.